

Practical 5

Problem definition:

Perform Create, Read, Update, and Delete (CRUD) operations using MongoDB. Students will work with a NoSQL document database to understand how unstructured and semi-structured data is managed at scale in Big Data environments.

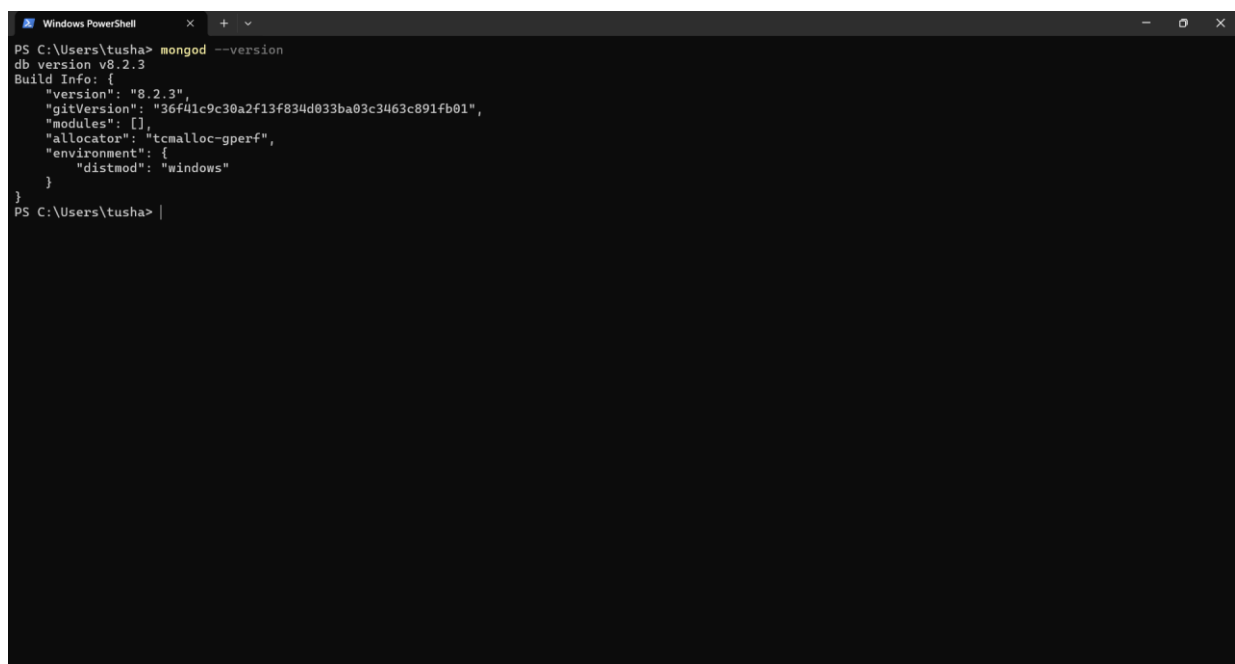
Objectives:

1. To upload the NCDC weather dataset into HDFS.
2. To examine and understand the structure of NCDC weather records.
3. To develop a Mapper program for extracting the year and temperature values.
4. To generate intermediate key-value pairs in the form (Year, Temperature).
5. To develop a Reducer program for finding the maximum temperature for each year.
6. To execute the MapReduce job on Hadoop.
7. To store and verify the generated output in HDFS.
8. To analyze yearly maximum temperature trends.
9. To identify the hottest year in the dataset.
10. To understand the advantages of MapReduce for large-scale climate data analysis.

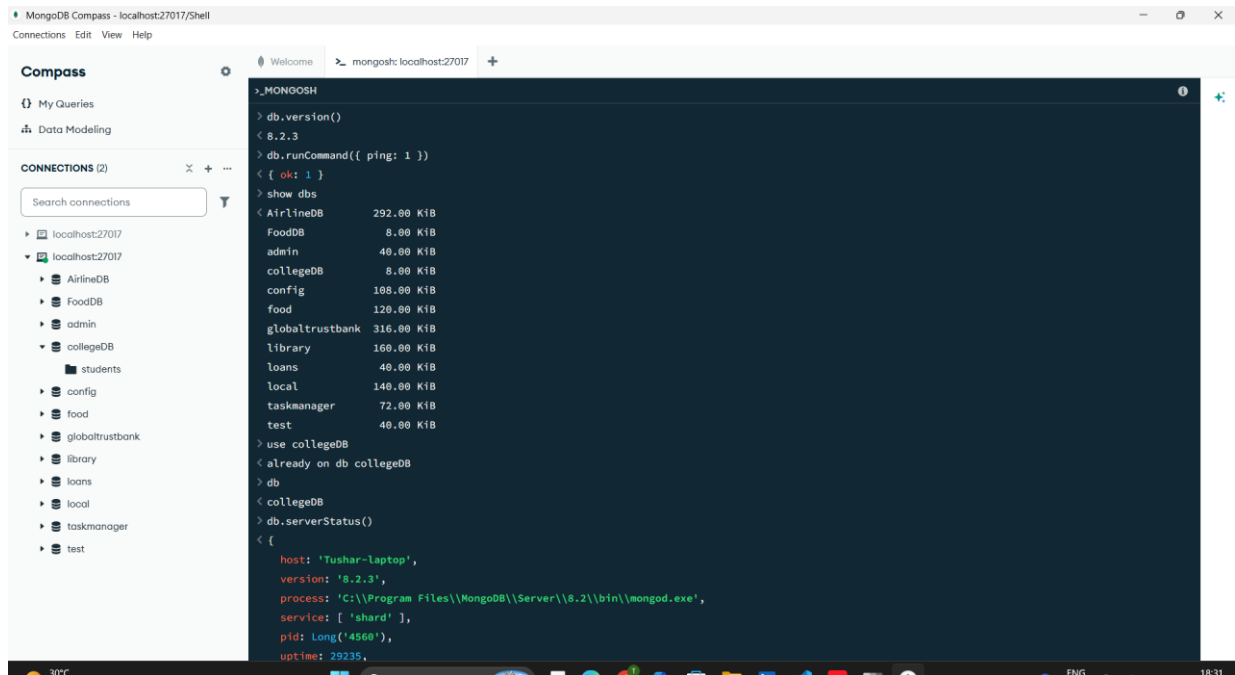
Observation / Output:

Task 1: Install and configure MongoDB.

```
mongod --version
db.version()
db.runCommand({ ping: 1 })
show dbs
use collegeDB
db
db.serverStatus()
```



```
Windows PowerShell
PS C:\Users\tusha> mongod --version
db version v0.2.3
Build Info: {
  "version": "0.2.3",
  "gitVersion": "36f41c9c30a2f13f834d033ba03c3463c891fb01",
  "modules": [],
  "allocator": "tcmalloc-gperf",
  "environment": {
    "distmod": "windows"
  }
}
PS C:\Users\tusha> |
```



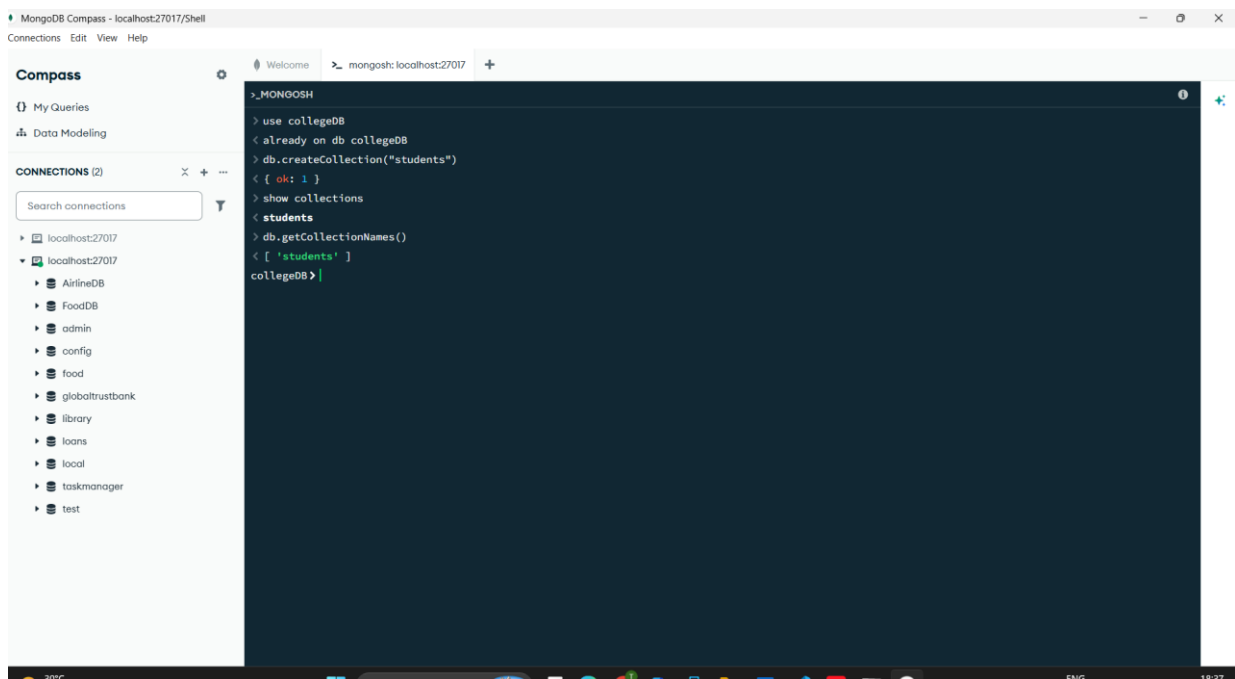
Task 2: Create a database and collection.

use collegeDB

db.createCollection("students")

show collections

db.getCollectionNames()

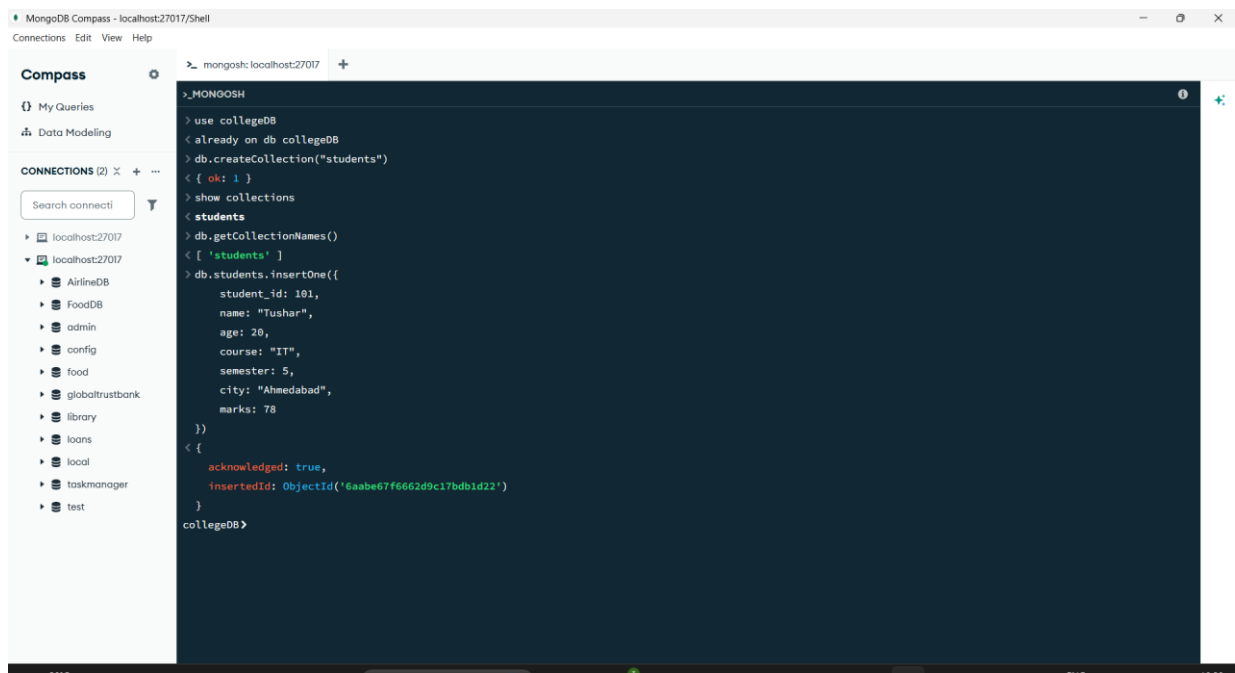


Task 3: Insert documents (single and multiple).

```
db.students.insertOne({
  student_id: 101,
  name: "Tushar",
  age: 20,
  course: "IT",
  semester: 5,
  city: "Ahmedabad",
```

marks: 78

})

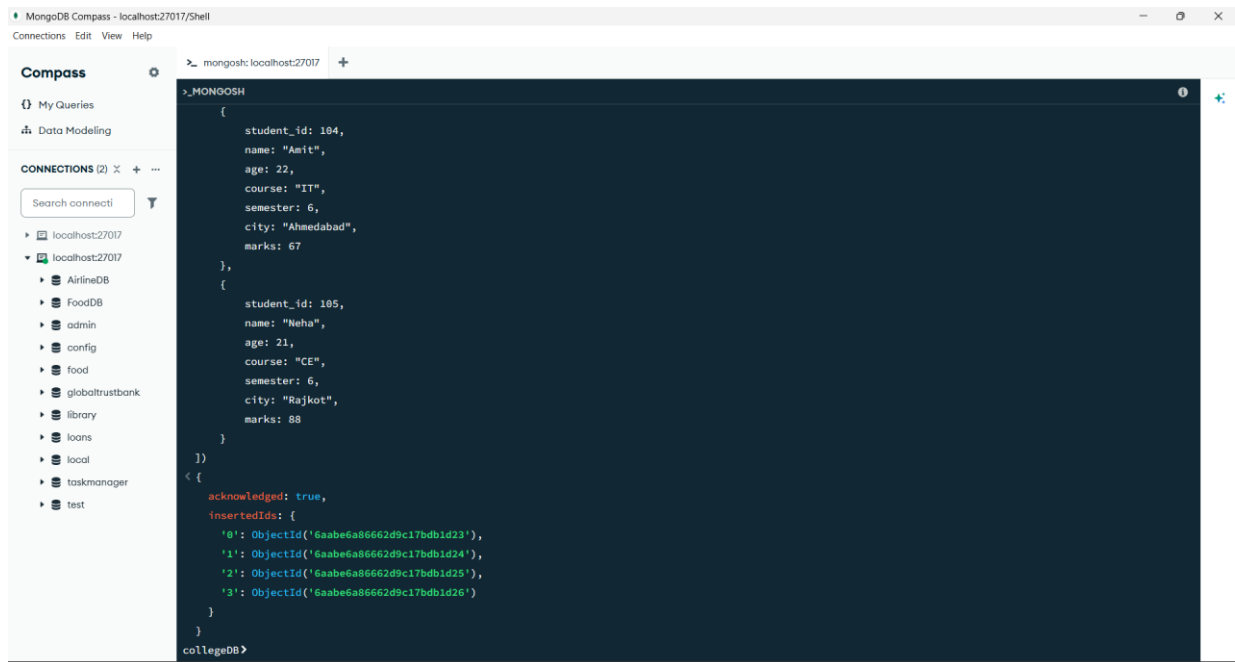


```

db.students.insertMany([
  {
    student_id: 102,
    name: "Rahul",
    age: 21,
    course: "IT",
    semester: 5,
    city: "Vadodara",
    marks: 85
  },
  {
    student_id: 103,
    name: "Priya",
    age: 20,
    course: "CE",
    semester: 5,
    city: "Surat",
    marks: 91
  },
  {
    student_id: 104,
    name: "Amit",
    age: 22,
    course: "IT",
    semester: 6,
    city: "Ahmedabad",
    marks: 67
  },
  {
    student_id: 105,
    name: "Neha",
    age: 21,
    course: "CE",
    semester: 6,
    city: "Rajkot",

```

```
marks: 88
    }
  ]
}
```

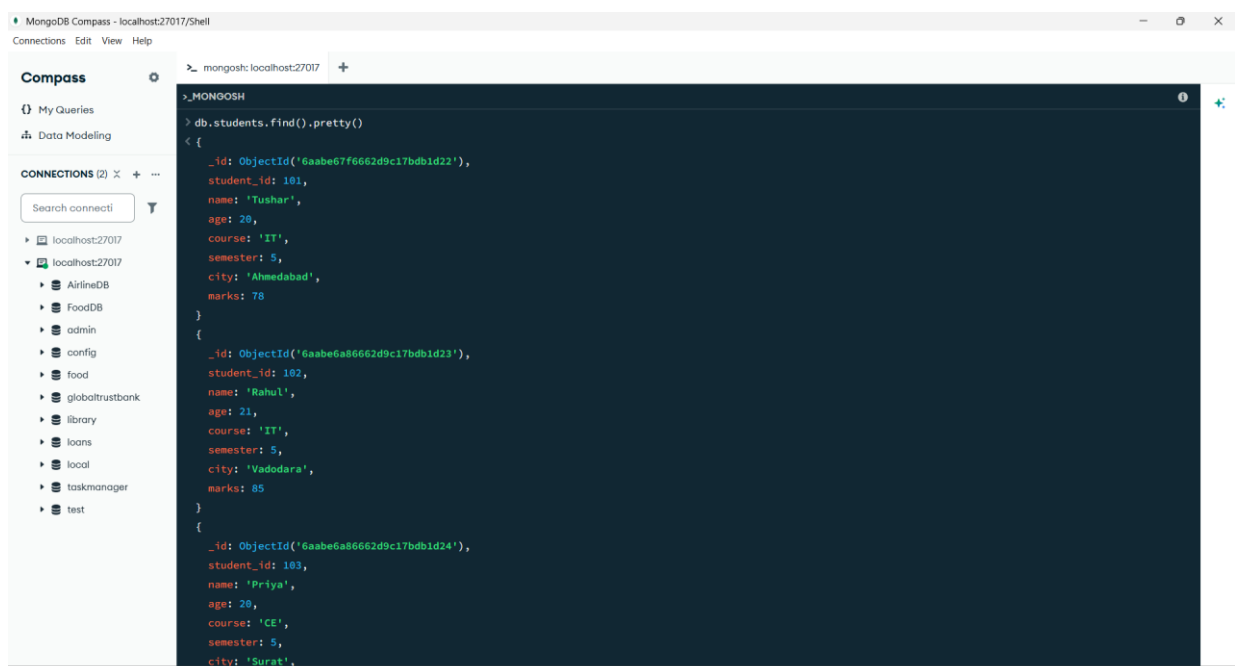


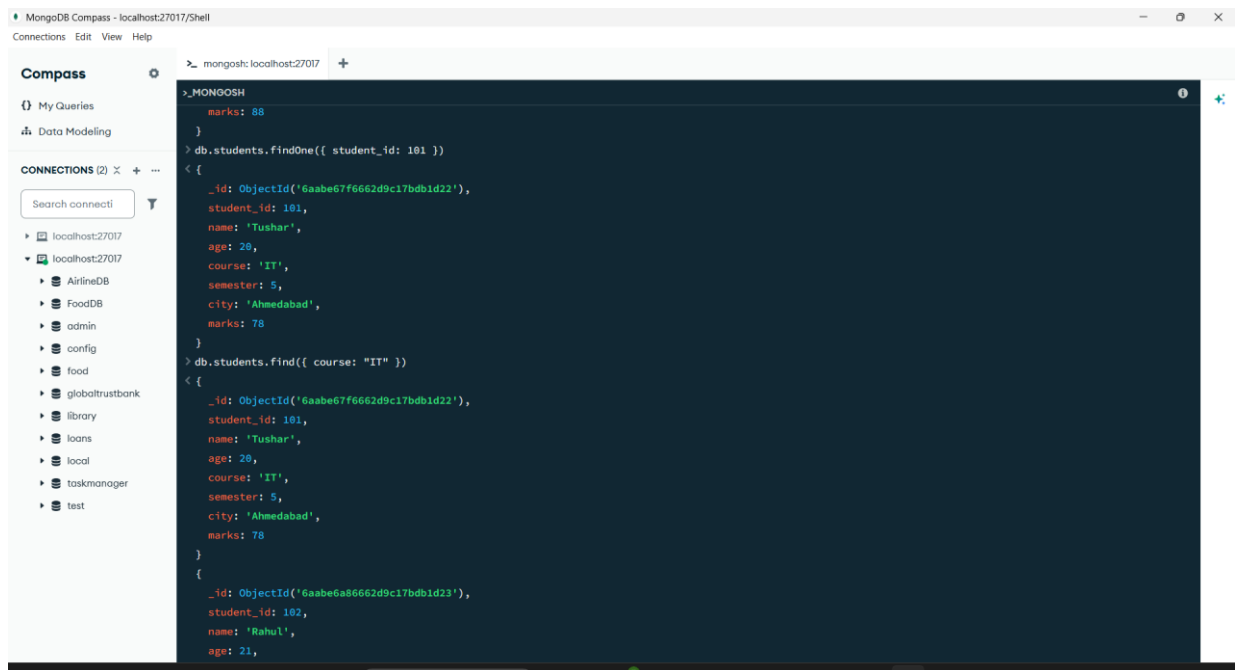
Task 4: Read/Query documents using various filters.

```
db.students.find().pretty()
```

```
db.students.findOne({ student_id: 101 })
```

```
db.students.find({ course: "IT" })
```

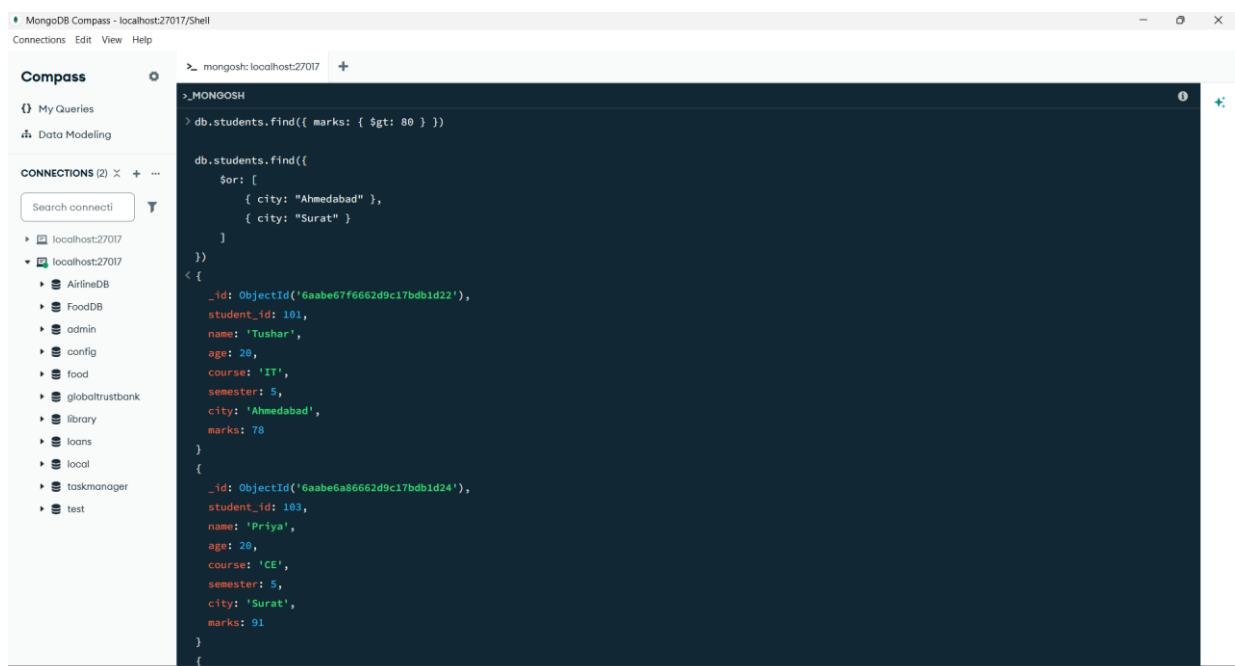


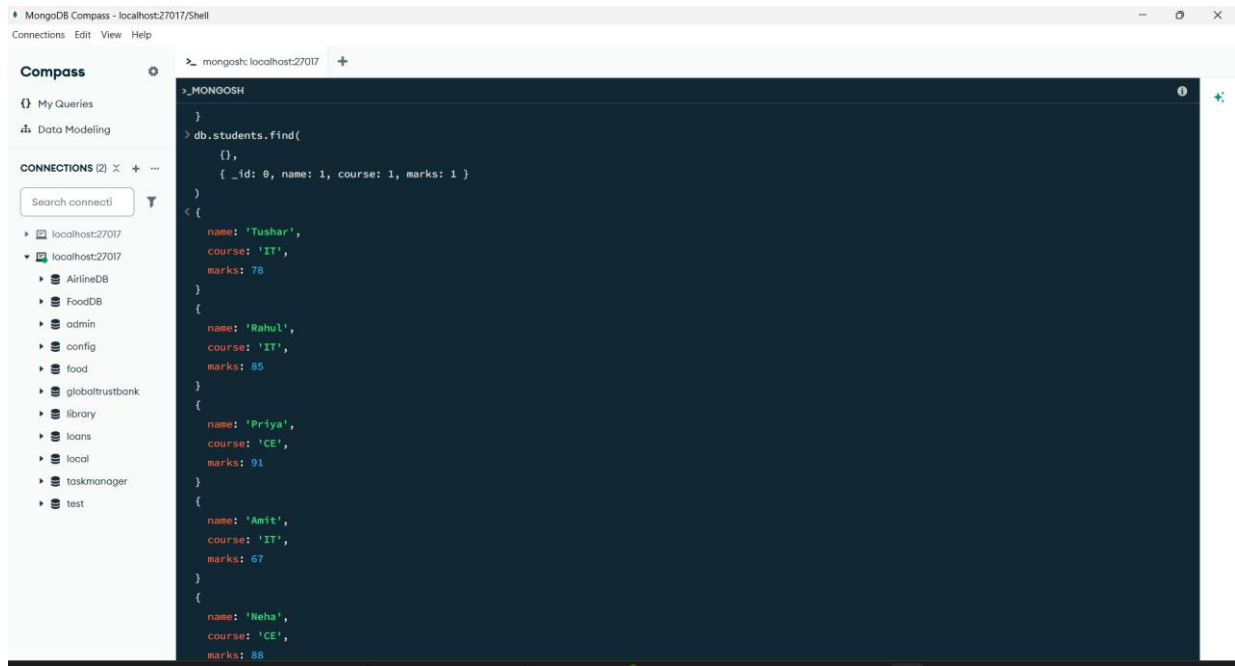


```
db.students.find({ marks: { $gt: 80 } })
```

```
db.students.find({
  $or: [
    { city: "Ahmedabad" },
    { city: "Surat" }
  ]
})
```

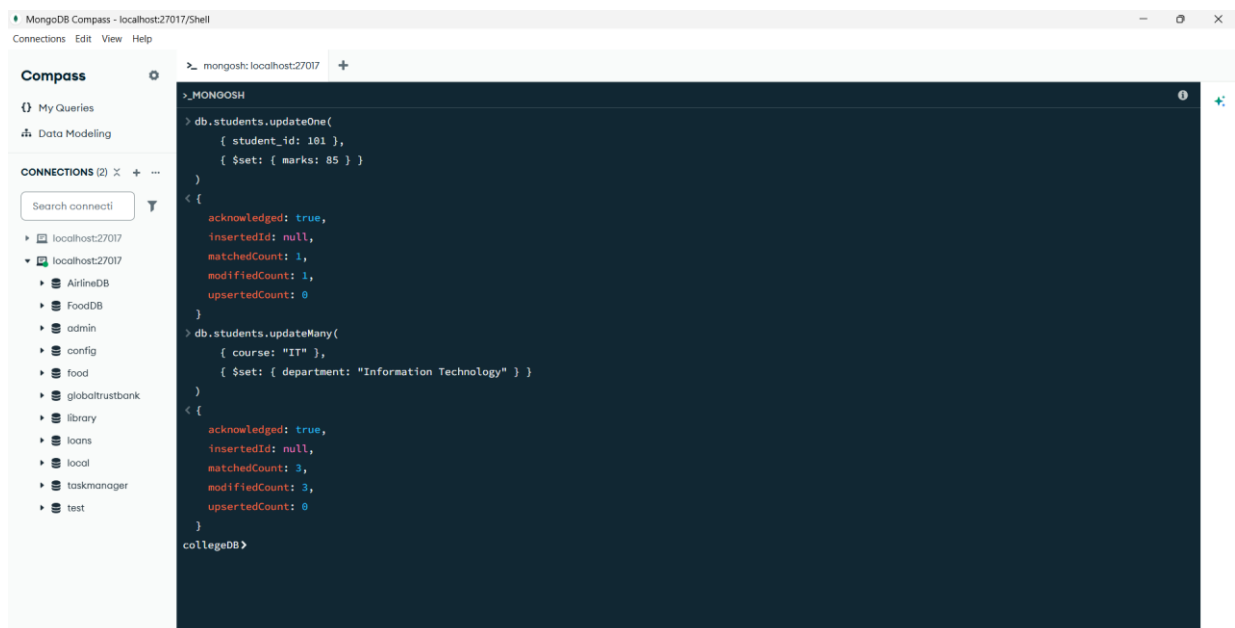
```
db.students.find(
  {},
  { _id: 0, name: 1, course: 1, marks: 1 }
)
```



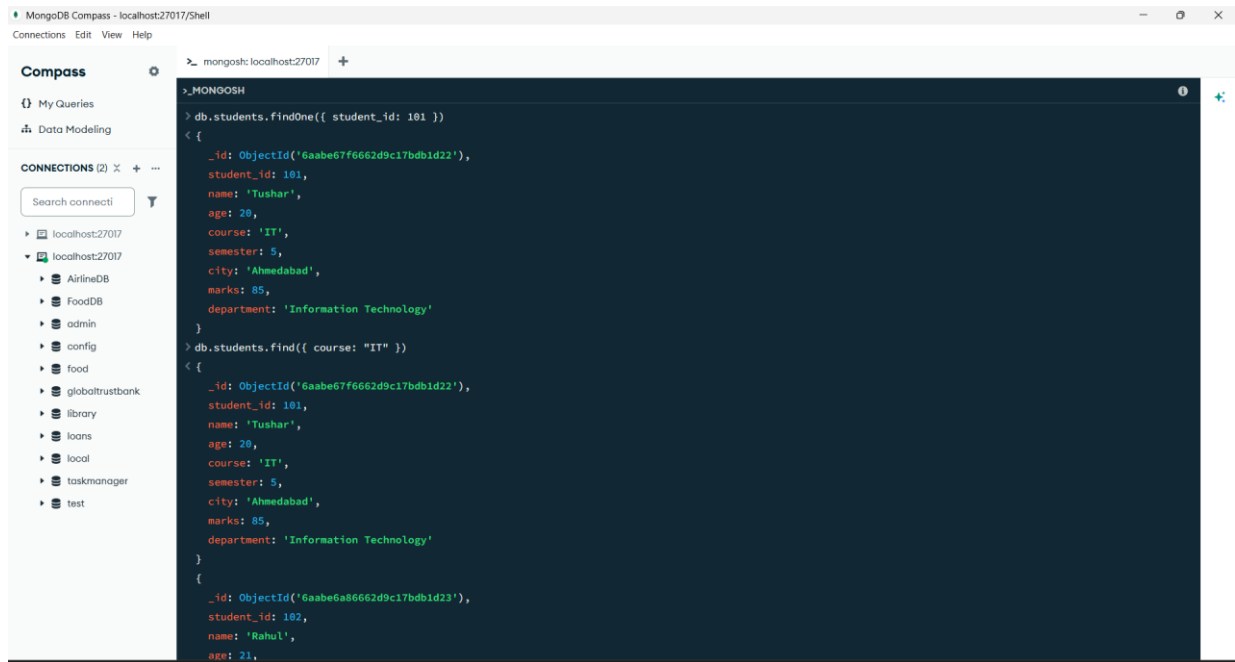


Task 5: Update existing documents.

```
db.students.updateOne(  
  { student_id: 101 },  
  { $set: { marks: 85 } }  
)  
db.students.updateMany(  
  { course: "IT" },  
  { $set: { department: "Information Technology" } }  
)
```

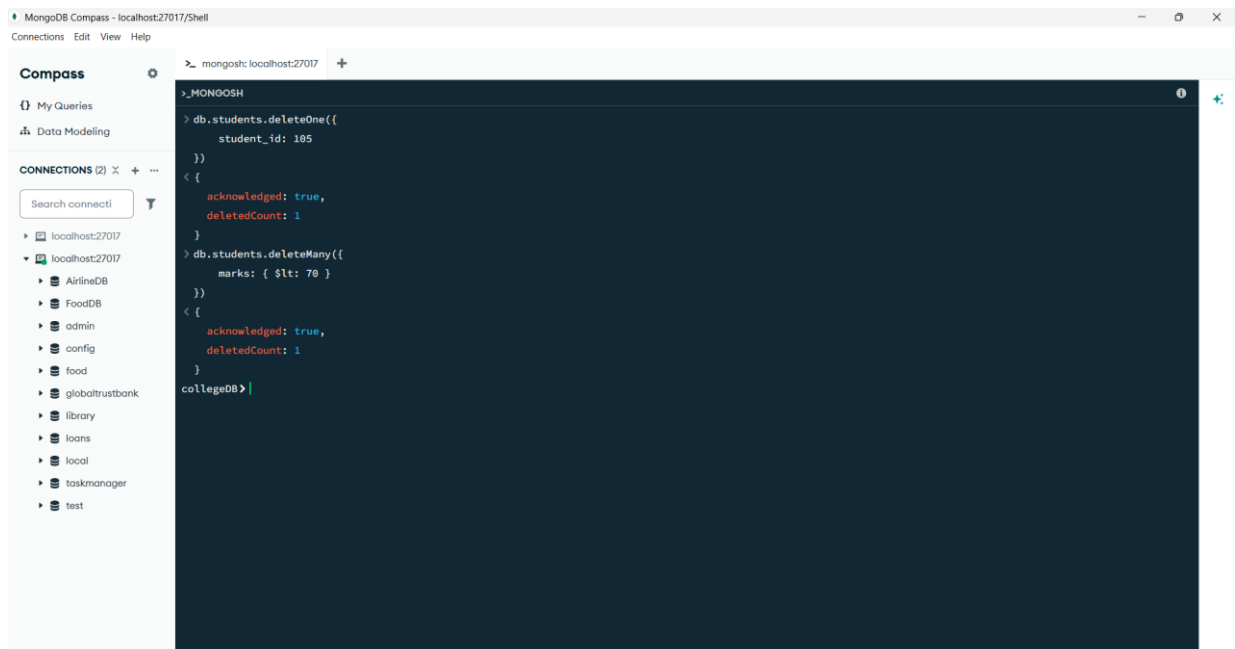


```
db.students.findOne({ student_id: 101 })  
db.students.find({ course: "IT" })
```

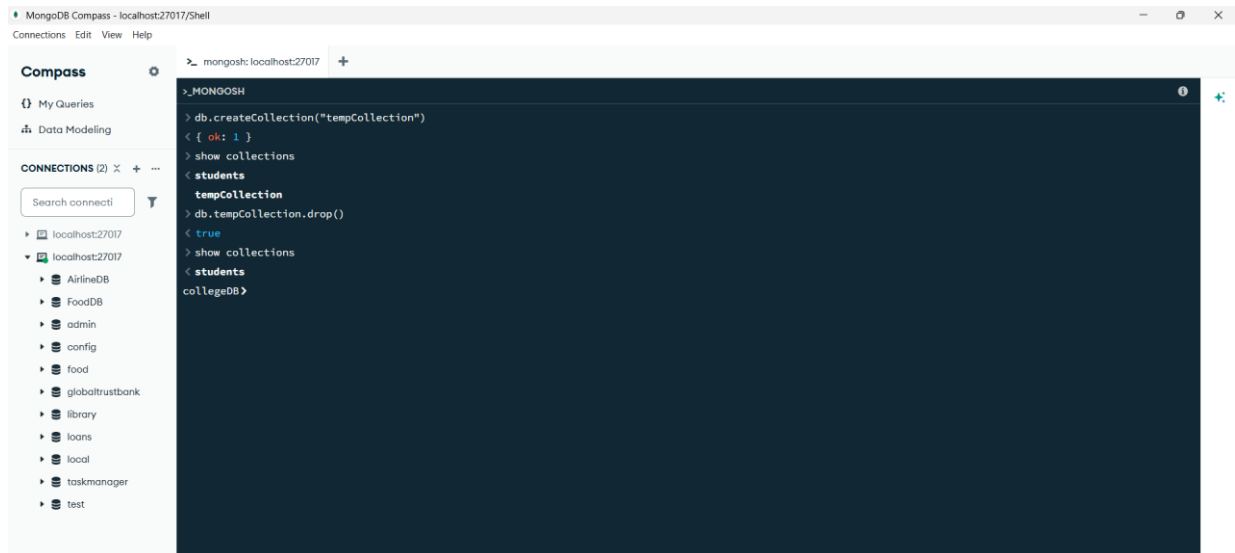


Task 6: Delete documents and collections.

```
db.students.deleteOne({
  student_id: 105
})
db.students.deleteMany({
  marks: { $lt: 70 }
})
```



```
db.createCollection("tempCollection")
db.tempCollection.drop()
show collections
```



Task 7: Use MongoDB aggregation pipeline for data analysis.

```
db.students.aggregate([
  {
    $group: {
      _id: "$course",
      totalStudents: { $sum: 1 }
    }
  }
])
```

```
db.students.aggregate([
  {
    $group: {
      _id: "$course",
      averageMarks: { $avg: "$marks" }
    }
  }
])
```

```
db.students.aggregate([
  {
    $group: {
      _id: null,
      highestMarks: { $max: "$marks" },
      lowestMarks: { $min: "$marks" }
    }
  }
])
```

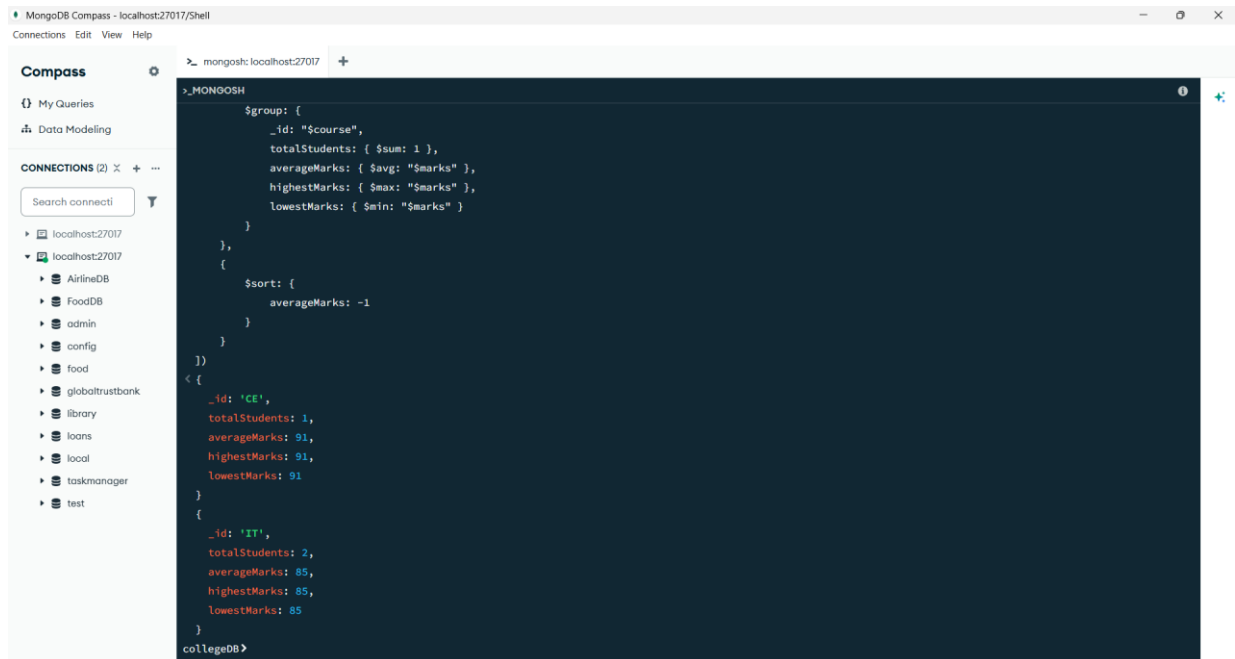
```
db.students.aggregate([
  {
    $group: {
      _id: "$course",
      totalStudents: { $sum: 1 },
```

```

        averageMarks: { $avg: "$marks" },
        highestMarks: { $max: "$marks" },
        lowestMarks: { $min: "$marks" }
    }
},
{
    $sort: {
        averageMarks: -1
    }
}
])

```

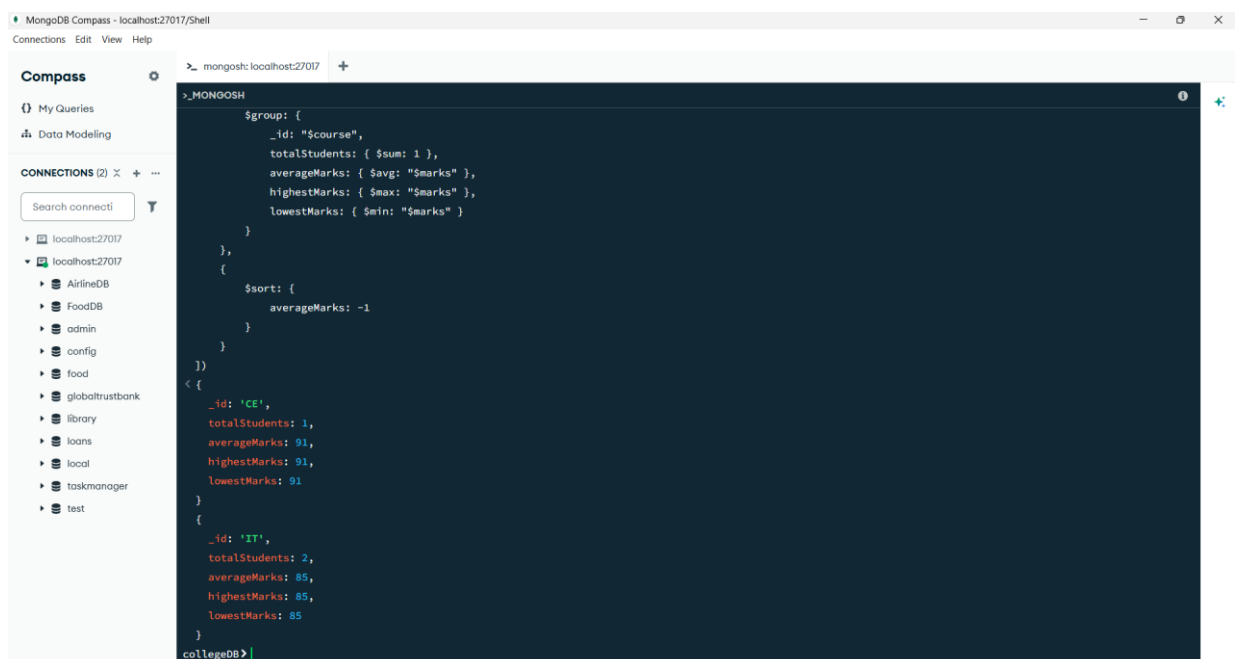




```

db.students.aggregate([
{
  $group: {
    _id: "$course",
    totalStudents: { $sum: 1 },
    averageMarks: { $avg: "$marks" },
    highestMarks: { $max: "$marks" },
    lowestMarks: { $min: "$marks" }
  },
},
{
  $sort: {
    averageMarks: -1
  }
}
])

```



Key Questions /Analysis:

Q1. What is MongoDB and how does it differ from relational databases?

MongoDB is a NoSQL, document-oriented database that stores data in flexible BSON documents. Unlike relational databases, MongoDB stores data in collections and documents instead of tables and rows. Relational databases usually use a fixed schema, while MongoDB supports a flexible schema. MongoDB can store nested documents and arrays directly within a document. Relational databases mainly use SQL and relationships/joins, whereas MongoDB uses document-based queries.

Q2. What are CRUD operations and why are they fundamental to database management?

CRUD stands for Create, Read, Update, and Delete, the four basic operations performed on database data. Create adds new records, Read retrieves existing records, Update modifies records, and Delete removes records. These operations are fundamental because almost every database application needs to manage its data in these four ways. They help maintain accurate, organized, and up-to-date information in a database. In MongoDB, CRUD operations are performed using methods such as `insertOne()`, `find()`, `updateOne()`, and `deleteOne()`.

Q3. How does MongoDB store data (BSON format)?

MongoDB stores data in BSON (Binary JSON) format, which is a binary representation of JSON-like documents. Each document contains key-value pairs and can store different data types such as strings, numbers, arrays, dates, and ObjectId. BSON allows MongoDB to efficiently store and process data. It also supports nested documents and arrays, making it suitable for semi-structured data. For example, a student document can contain fields like name, marks, and an array of skills.

Q4. What is the aggregation pipeline in MongoDB?

The aggregation pipeline in MongoDB is a framework used to process and analyze documents. It consists of multiple stages, where each stage performs a specific operation on the data. Common stages include `$match`, `$group`, `$sort`, `$project`, and `$limit`. It can be used to calculate averages, totals, maximum/minimum values, and summaries. For example, it can group students by course and calculate their average marks.

Q5. When should NoSQL databases be preferred over SQL databases?

NoSQL databases should be preferred when dealing with large-scale, unstructured or semi-structured data. They are useful when the data structure changes frequently and a flexible schema is required. NoSQL databases are also suitable for applications that need high scalability and fast data processing. They work well with distributed systems and Big Data applications. SQL databases are generally preferred when complex relationships, joins, and strict transactions are important.

Supplementary Problems:

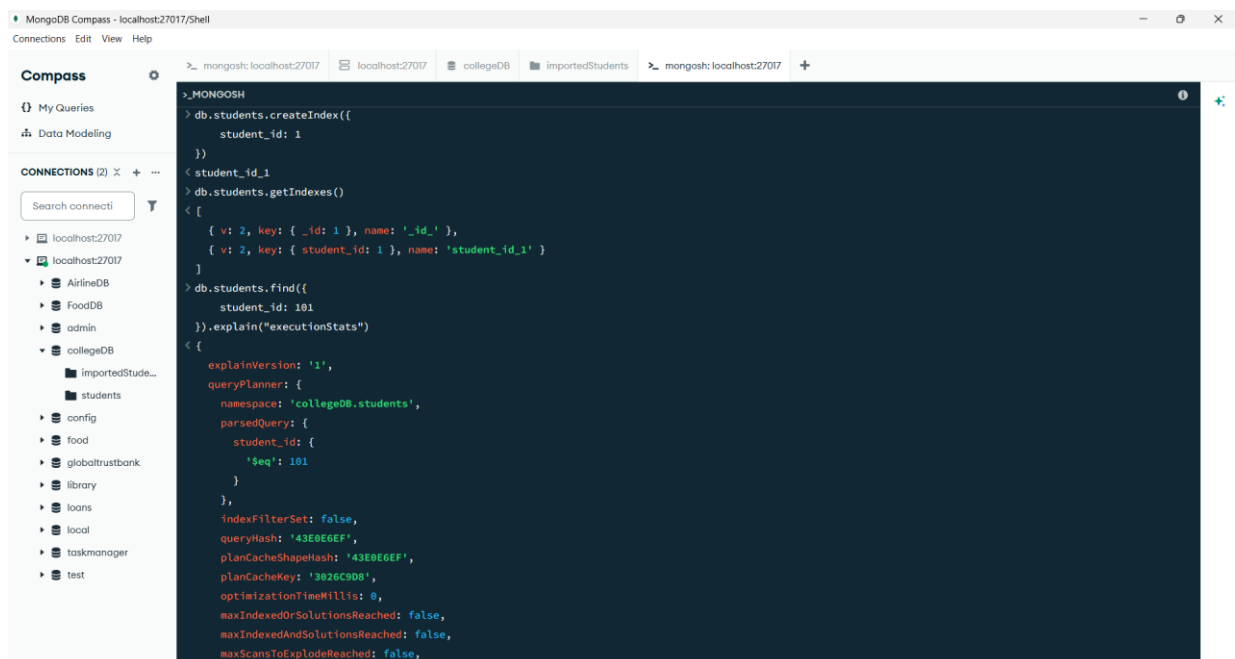
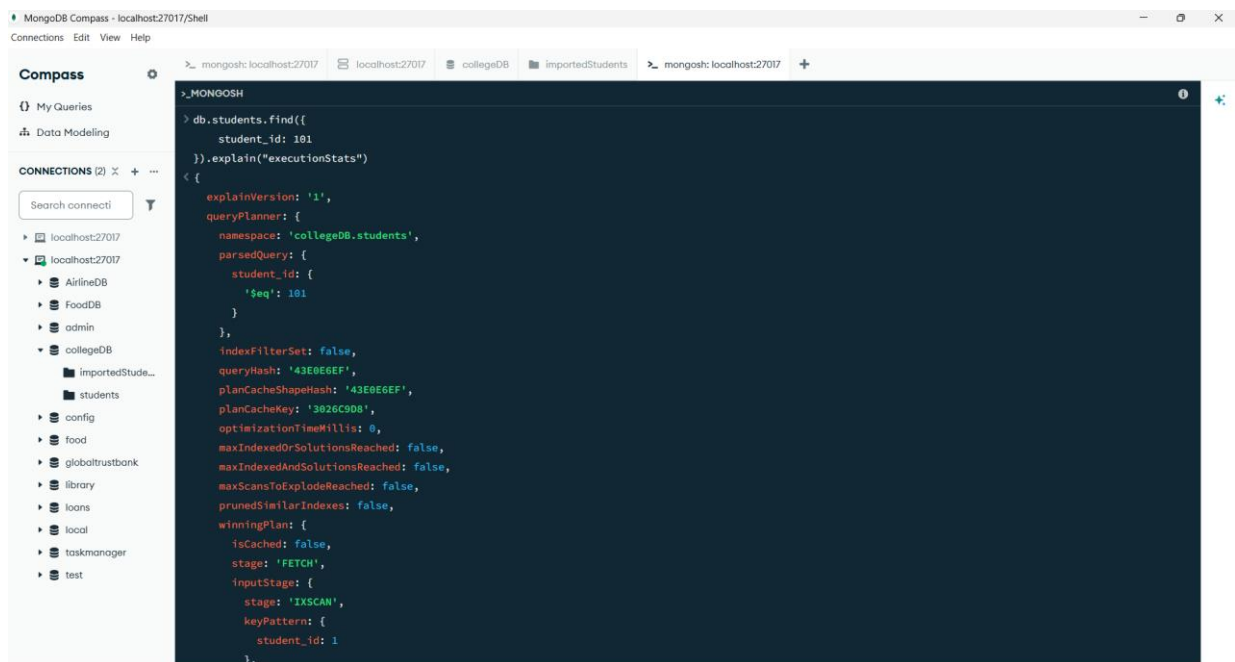
1. Create an index on a collection and compare query performance. Use the aggregation pipeline to generate a summary report. Import a JSON dataset into MongoDB and perform analytical queries.

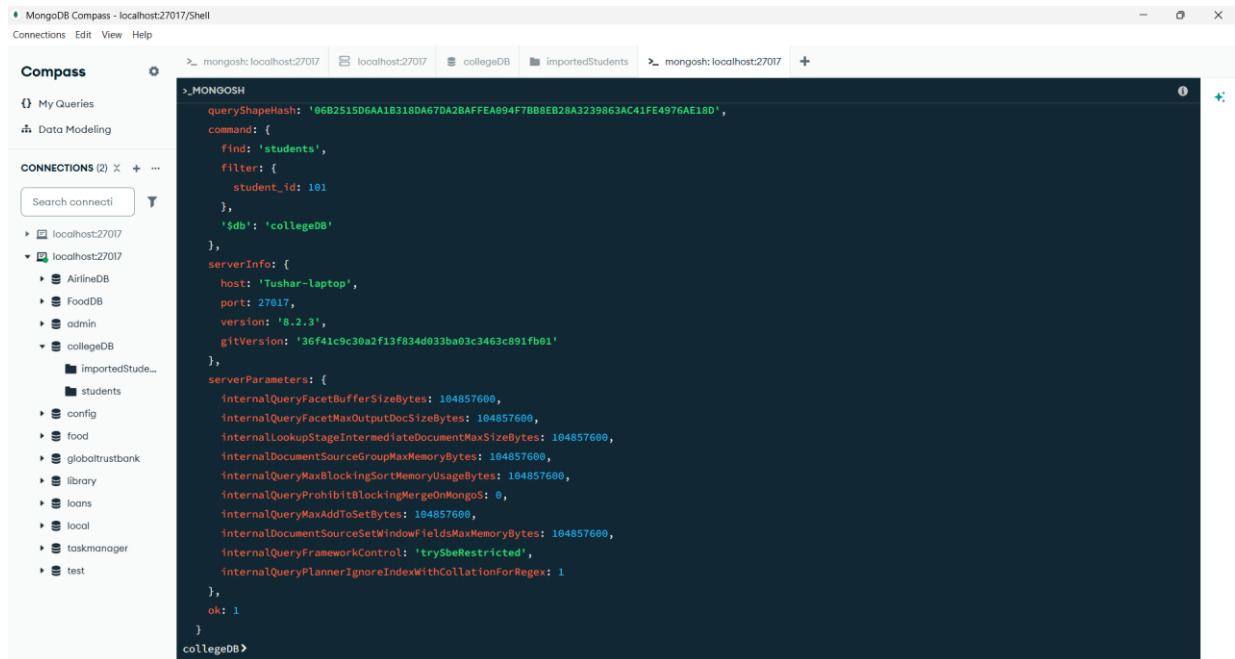
```
db.students.find({
  student_id: 101
}).explain("executionStats")
```

```
db.students.createIndex({
  student_id: 1
})
```

```
db.students.getIndexes()
```

```
db.students.find({
  student_id: 101
}).explain("executionStats")
```

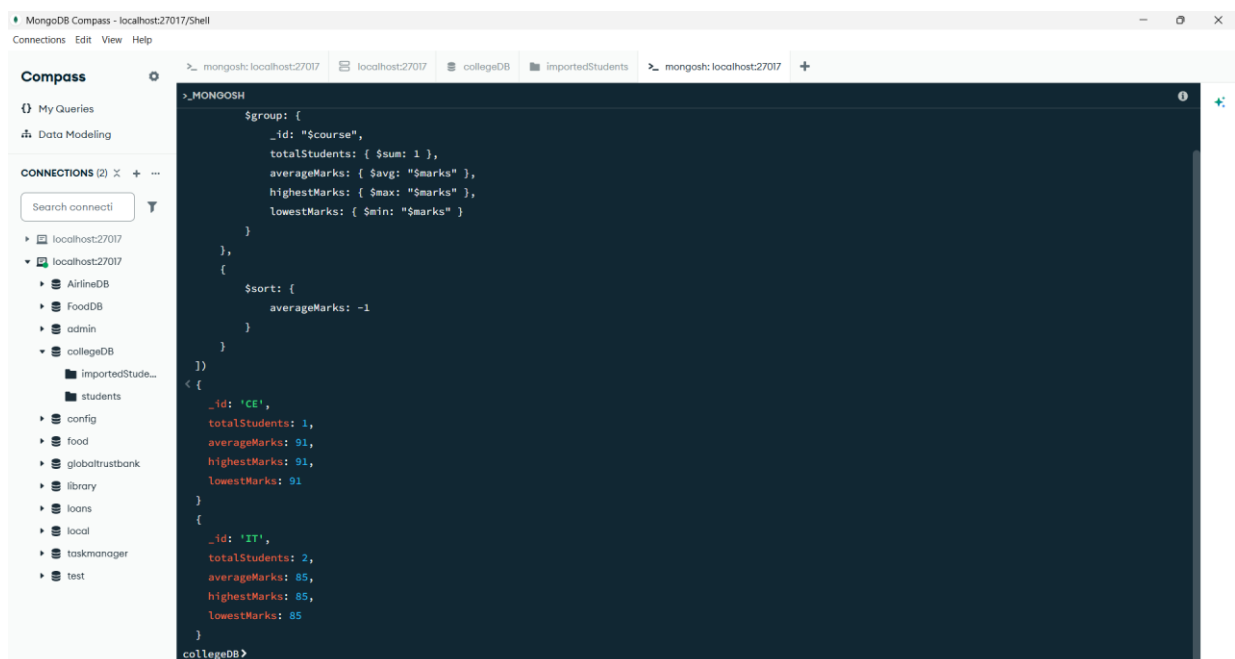




```

db.students.aggregate([
{
  $group: {
    _id: "$course",
    totalStudents: { $sum: 1 },
    averageMarks: { $avg: "$marks" },
    highestMarks: { $max: "$marks" },
    lowestMarks: { $min: "$marks" }
  }
},
{
  $sort: {
    averageMarks: -1
  }
}
])

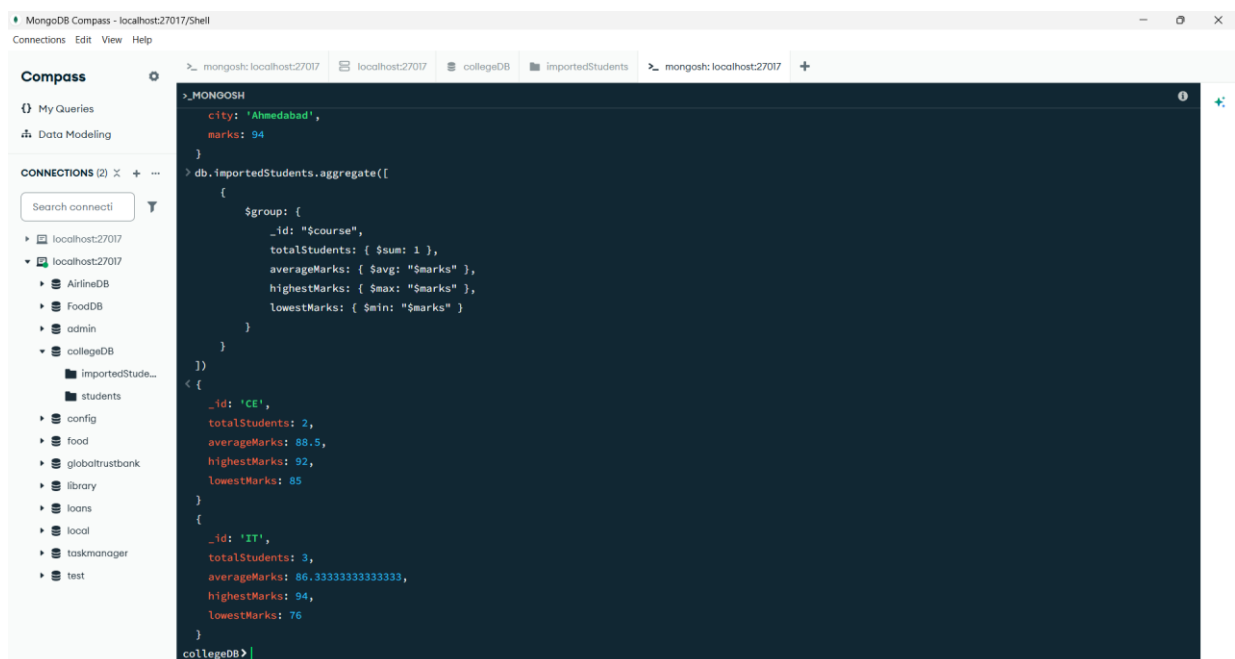
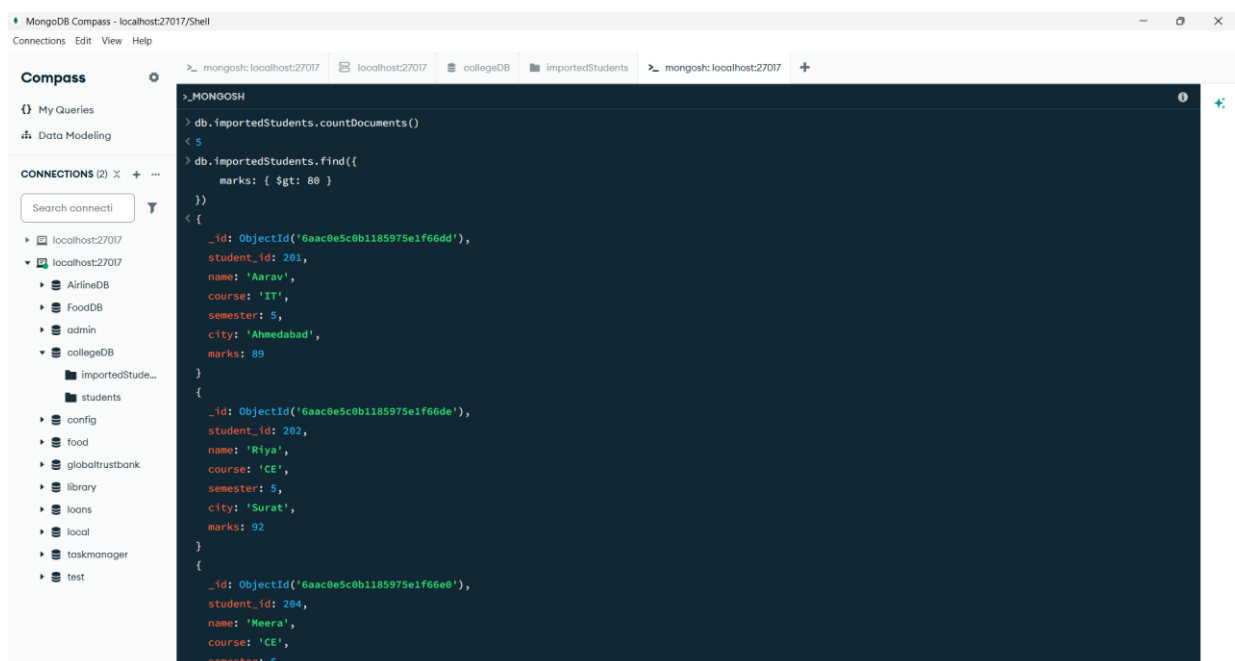
```



```

mongoimport --db collegeDB --collection importedStudents --file students.json --jsonArray
db.importedStudents.countDocuments()
db.importedStudents.find({
  marks: { $gt: 80 }
})
db.importedStudents.aggregate([
  {
    $group: {
      _id: "$course",
      totalStudents: { $sum: 1 },
      averageMarks: { $avg: "$marks" },
      highestMarks: { $max: "$marks" },
      lowestMarks: { $min: "$marks" }
    }
  }
])

```



Tools / Technology used:

MongoDB Installation & Configuration | CRUD Operations | NoSQL Data Modeling | Aggregation Pipeline | Database Management | MongoDB 6.x / 7.x | MongoDB Shell (mongosh) | MongoDB Compass (GUI) | VS Code

Conclusion

In this practical, MongoDB was successfully installed, configured, and connected using MongoDB Compass and MongoDB Shell. CRUD operations were performed to insert, retrieve, update, and delete student documents. Various filtering and querying techniques were used to retrieve data efficiently. The aggregation pipeline was used to generate useful summaries and analyze student data. An index was created to study query performance, and a JSON dataset was imported and analyzed. Overall, the practical provided a clear understanding of NoSQL document databases, flexible data modeling, CRUD operations, indexing, and data analysis using MongoDB.

Github Repo:

<https://github.com/tushar-2606/BDA-SHP-2026>